

Mini-projet 06 — Ta première vraie classe : Cible

Pourquoi ce mini-projet ?

Depuis le début tu représentes une cible par un dict : `{"ip": "10.0.0.5", "port": 22}`. Ça marche, mais ça ne **connaît** rien : ça ne sait pas s'afficher, se comparer, se valider. Une **classe** transforme ce sac de données en un vrai **objet** qui a des données ET des comportements. C'est la fondation de toute ta boîte à outils : `Cible`, `Plugin`, `Rapport`... tout sera des classes.

1. Classe, objet, instance

Terme	Analogie	En code
Classe	le plan d'une maison	<code>class Cible:</code>
Instance / objet	une maison construite d'après le plan	<code>Cible("10.0.0.5", 22)</code>

Une classe, c'est un moule. Chaque appel `Cible(...)` fabrique un nouvel objet indépendant.

2. __init__ et self

```
class Cible:
    def __init__(self, ip, port):
        self.ip = ip          # on RANGE ip dans l'objet
        self.port = port      # on RANGE port dans l'objet
```

- `__init__` = le **constructeur**. Il s'exécute automatiquement à chaque `Cible(...)`. Son rôle : **ranger les données dans l'objet**.
- `self` = **l'objet en train d'être construit**. C'est « moi-même ». `self.ip = ip` veut dire « range le paramètre `ip` dans mon attribut `ip` ».

Le **double underscore** autour de `__init__` (« dunder », pour *double under*) signale une méthode **spéciale** que Python appelle tout seul à des moments précis. Tu n'écris jamais `objet.__init__()` toi-même.

```
c = Cible("10.0.0.5", 22)  # __init__ s'exécute : self.ip="10.0.0.5", self.port=22
print(c.ip)               # 10.0.0.5    <- on lit l'attribut avec un POINT
print(c.port)             # 22
```

⚠ **Piège — un objet n'est PAS une liste.** On accède aux données d'un objet par `.attribut` (avec un point), **jamais** par `[0]` :

```
c[0]          # ✖ TypeError: 'Cible' object is not subscriptable
c.ip          # ✔ la bonne façon

# Et on ne dépaquette pas un objet comme un tuple :
ip, port = c  # ✖ erreur — c n'est pas une séquence
ip = c.ip     # ✔
```

3. Les méthodes

Une méthode = une fonction **définie dans la classe**, qui agit sur l'objet. Son premier paramètre est **toujours** `self`.

```
class Cible:
    def __init__(self, ip, port):
        self.ip = ip
        self.port = port

    def adresse(self):          # méthode : elle utilise les données de l'objet
        return f"{self.ip}:{self.port}"

c = Cible("10.0.0.5", 22)
print(c.adresse())            # 10.0.0.5:22
```

Pourquoi `self` partout ? Quand tu écris `c.adresse()`, Python appelle en réalité `Cible.adresse(c)` : il passe `c` comme `self`. C'est comme ça que la méthode sait sur QUEL objet travailler.

4. `__repr__` — un affichage lisible

Par défaut, afficher un objet donne un truc moche :

```
print(c) # <__main__.Cible object at 0x7f3a...> illisible !
```

La méthode spéciale `__repr__` définit ce qui s'affiche. Elle doit **renvoyer une chaîne** :

```
def __repr__(self):
    return f"Cible({self.ip}:{self.port})"

print(c) # Cible(10.0.0.5:22) <- lisible !
```

Bonus pro : `__repr__` sert aussi quand tu affiches une **liste d'objets** — chaque élément utilise son `__repr__`. Très pratique pour déboguer.

5. `__eq__` — comparer deux objets notion clé

Par défaut, deux objets **différents** ne sont jamais égaux, même s'ils contiennent la même chose :

```
a = Cible("10.0.0.5", 22)
b = Cible("10.0.0.5", 22)
a == b # False (!) - Python compare les EMPLACEMENTS mémoire, pas le contenu
```

Pour que « même IP + même port = égal », on définit `__eq__` :

```
def __eq__(self, autre):
    return self.ip == autre.ip and self.port == autre.port

a == b # True maintenant
```

Le lien avec tes anciens bugs : avant tu comparais un dict à une str et ça faisait des surprises. Ici, `__eq__` te laisse décider **toi-même** ce que « égal » veut dire pour tes objets. C'est toi qui contrôles.

🎯 Ta mission

Objectif : écrire la classe `Cible` complète, avec constructeur, méthode, affichage et comparaison.

Ta classe doit avoir :

- `__init__(self, ip, port)` → range `ip` et `port` dans l'objet. **Convertis le port en `int`** au passage (réflexe MP01).
- `adresse(self)` → renvoie la chaîne `"ip:port"`.
- `__repr__(self)` → renvoie un affichage lisible, ex. `Cible(10.0.0.5:22)`.
- `__eq__(self, autre)` → deux cibles sont égales si même IP ET même port.

Tests à faire passer :

```
a = Cible("10.0.0.5", "22") # note : port passé en str
print(a) # -> Cible(10.0.0.5:22)
print(a.adresse()) # -> 10.0.0.5:22
print(a.port, type(a.port)) # -> 22 (bien converti !)

b = Cible("10.0.0.5", 22)
print(a == b) # -> True

c = Cible("10.0.0.5", 80)
print(a == c) # -> False

cibles = [a, b, c]
print(cibles) # -> [Cible(10.0.0.5:22), Cible(10.0.0.5:22), Cible(10.0.0.5:80)]
```

Bonus : ajoute une méthode `est_bien_connu(self)` qui renvoie `True` si le port est `< 1024` (les « ports bien connus » : 22, 80, 443...).

✓ Critères de réussite

- `a.port` est un `int` même quand on passe `"22"` .
- `print(a)` affiche `Cible(10.0.0.5:22)` , pas l'adresse mémoire.
- `a == b` vaut `True` , `a == c` vaut `False` .
- Afficher la liste `cibles` montre les 3 en lisible (grâce à `__repr__`).

Si tu bloques — indices progressifs

Étape	Indice
Structure	Commence par le squelette : <code>class Cible:</code> puis <code>def __init__(self, ip, port):</code> . N'oublie aucun <code>self</code> .
Port en int	Dans <code>__init__</code> : <code>self.port = int(port)</code> (pas juste <code>port</code>).
adresse	<code>return f"{self.ip}:{self.port}"</code> — n'oublie pas les <code>self.</code> , sinon Python ne trouve pas les variables.
<code>__eq__</code>	Deux conditions reliées par <code>and</code> , en comparant <code>self.xxx</code> à <code>autre.xxx</code> .

Où ranger : `mini-projets/06-classe-cible/cible.py` .

Astuce : cette classe `Cible` , tu vas la **réutiliser** dans les mini-projets suivants (7, 8, et au Module réseau). Soigne-la, c'est une pièce maîtresse de ton grand projet.

Quand c'est prêt : `git add .` → `commit` → `push` , puis « c'est poussé ».